HACKATHON MVP

# IRYS

AI governance & server access — one control plane for provider organisations.

dashboard | mcp server | cli | ssh agent

governance files · scoped ssh keys · deployment procedures · audit

THE PROBLEM

# AI agents ship faster than governance.

The rules exist — but they never travel with the work.

- Compliance lives in wikis and senior engineers' heads — not in the tools people actually use.
- Even experienced engineers forget a crucial control; interns and new hires never learned them.
- Server access is shared keys and stale authorized_keys — no per-project scope, no revocation.
- Your AI agent knows none of your standards — so it cannot possibly follow them.

**0**

source of truth for AI rules

`the agent improvises`

**N×**

checklists redone per dev

`same rules, re-litigated`

**?**

who had prod access last month

`no scope, no audit`

THE IDEA

# Define once. Enforce everywhere.

One source of truth for agent behaviour AND server access — not another policy doc.

1

## Define



Describe the project in plain language, or write rules by hand. Toggles, compliance checklists and quality bars become the source of truth.

2

## Distribute



IRYS generates AGENTS.md / COMMANDCODE.md / IRYS.md. CLI syncs them into repos; MCP serves them to any AI coding harness.

3

## Enforce



Scoped SSH keys reach only in-scope servers. Deployments follow the reviewed procedure — and every action is audited.

`dashboard → governance files + scope → cli / mcp → agent ssh → scoped server`

## AI governance files

AGENTS.md / COMMANDCODE.md / IRYS.md generated from toggles, compliance & quality controls — describe mode or fully manual.

## Scoped SSH access

Members scoped project-by-project. authorized_keys recomputed per scope; keys added and removed from the dashboard.

## Deployment procedures

Free-text guideline + optional steps → generated bash script and systemd unit at /services/{project}.

## Audit everything

Logins, key syncs, scope changes, deploys — every governed action recorded and queryable via CLI / MCP.

HOW IT WORKS

# Keys pull. Deploys act.

IRYS never SSHes into your hosts to manage keys — but the agent it keys CAN ssh in to deploy.

plane 1 — key sync · pull-based

1 **REGISTER**
Admin adds a server in the dashboard → one-time bootstrap token issued.

→

2 **BOOTSTRAP**
One curl command on the host: creates the irys user, links, installs a 60s timer.

→

3 **PULL**
Agent pulls authorized_keys per scope. IRYS never needs inbound SSH.

plane 2 — deployment · your agent acts

1 **KEYED**
The agent holds an SSH key IRYS provisioned

→

2 **SSH IN**
Agent SSHes into the scoped server with that

→

3 **APPLY**
git pull → build → /services/{project} →

IRYS doesn't SSH in — your agent does. Scoped, keyed and audited by IRYS.

LIVE DEMO

# Five minutes. Five proofs.

Each beat maps to a step in the demo runbook.

**1 LOGIN**

`irys login`

Local SSH key discovered, fingerprinted, auto-registered to scope.

**2 GOVERN**

`describe → AGENTS.md`

Real rules, GDPR checklist, quality bar — generated live.

**3 CLI + MCP**

`same file, twice`

CLI and MCP return the exact governance the provider authored.

**4 SCOPE**

`keys recompute`

Add server to scope → authorized_keys rebuilt for that scope only.

**5 AUDIT**

`full trail`

Every step above lands in the audit log — queryable, attributable.

~25s each · dashboard + terminal side by side · fallbacks in the runbook

FOR UNIVERSITIES

# Set the standard once — every project inherits it.

- Professors define what students CAN and CANNOT do on each assigned project — inside the tools students already use.
- Code-quality bars, submission rules and compliance become the project's AGENTS.md — the student's AI follows them by default.
- No more "the agent wrote it, not me": every agent action on lab infra is scoped and audited.
- Per-team SSH scope on shared lab servers — granted at kickoff, revoked at the deadline.
- TAs stop policing the same ten violations; the standard is encoded once, enforced everywhere.

course setup

CS-3010 · capstone

PROJECT
atlas-platform

SCOPE
team 4 (5 students)

SERVER
lab-staging (scoped)

EXPIRY
scope ends May 12

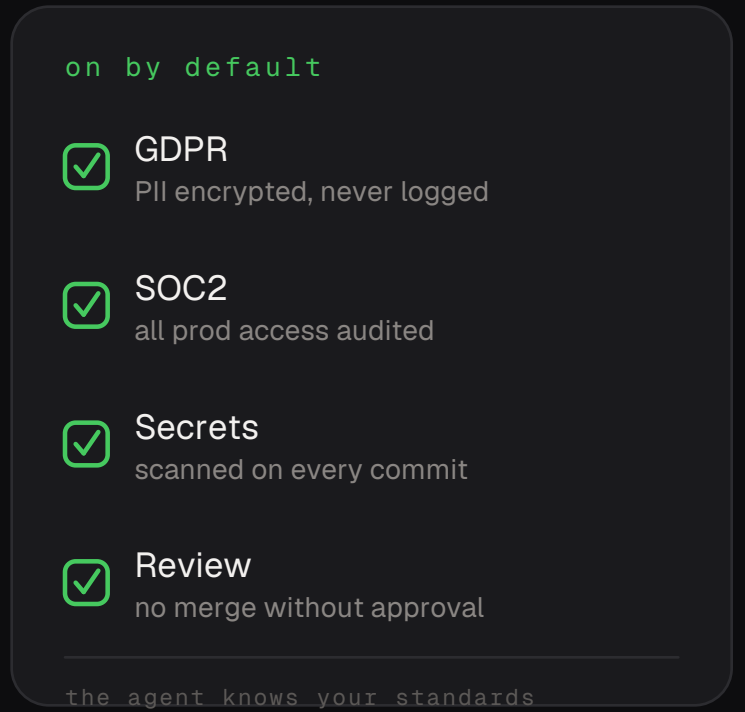
rules

- no prod writes without CI
- PII encrypted at rest
- 80% test coverage

FOR SMALL BUSINESSES

# Compliance by default — nobody has to remember it.

- GDPR, secret-handling and PII rules are baked into every agent by default — not re-checked per developer, per ticket.
- Experienced devs forget crucial controls too; new interns never knew them. With IRYS they don't have to think about it.
- The agent already knows what the company needs, wants and prefers — from IRYS instructions, guidelines and documents.
- Day-one productivity: a new hire's AI is onboarded the moment they log in through the CLI.
- Scoped server access replaces shared root keys — remove someone, their access disappears with them.



UNDER THE HOOD

# Built lean. Shown honestly.

| | |
|---|---|
| API & DASHBOARD | Node + Express · JSON store · JWT/bcrypt · React 18 SPA (uwitz design system) |
| MCP SERVER | 10 tools + governance resources over stdio — Claude Code, Cursor, CodeBuddy connect as-is |
| CLI | irys login / sync-key / governance / sync / deploy / scope / audit — same shared client |
| KEY AGENT | POSIX bash · systemd timer (cron fallback) · host pulls keys — no inbound SSH from IRYS |
| DEPLOY | your agent SSHes in with the IRYS-provisioned key, runs the reviewed procedure, audits it |



# Thank you.

Pilots wanted: one course, one small team. If your agents ship to real servers — let's talk.

`localhost:4000` `irys cli + mcp` `demo runbook`